



FOR JAVA DEVELOPERS PORTING A CONTROLLER

Right Route. Wrong Handler.

Spring Boot to NestJS, episode 2

It compiles, it starts, the smoke test passes, and two things are already wrong.

A faithful port is the dangerous kind. You go annotation by annotation, the decorators line up, the compiler is happy and the application starts. Nothing in that process tells you that two parts of the HTTP response are now decided differently.

Both bugs on this sheet return 200 or 201. Neither crashes. Neither is caught by a compiler, a linter, or a smoke test that only ever asks for a payment by its id.

The versions on this sheet were current on 22 September 2026: NestJS 12, Express 5 as the default adapter, Fastify 5 as the alternative, and Spring Boot 4. The default status code per verb and the adapter chosen by default are both release decisions, so check the date on anything else you read about them.

In scope

- What `@Controller` means in each framework, and what `@ResponseBody` was doing
- Which Spring web annotations transfer, and which only look like they do
- Route declaration order, and why it changes the answer
- The HTTP adapter, and why the routing rule is a fact about it
- The default status code, which is per verb rather than per application
- Where `ResponseEntity` went, and what `@HttpCode` replaces it with
- The service, and what `@Service` was actually doing for you

Out of scope

- Validation and error handling, which have their own episode
- Guards, interceptors and pipes
- Persistence, which is covered in the flagship



One controller and one service, ported properly. Everything here is reproducible with a single script.

01 What the two stacks actually answered

GET /payments/recent, Spring	200 {"handler":"recent"}
GET /payments/recent, NestJS on Express	200 {"handler":"byId","id":"recent"}
GET /payments/recent, NestJS on Fastify	200 {"handler":"recent"}
POST /payments, Spring	200
POST /payments, NestJS	201
POST with @HttpCode(200), NestJS	200
Plain @Controller returning "hello", Spring	body empty
@RestController returning "hello", Spring	body hello
@Controller returning "hello", NestJS	body hello

Every row came off a terminal on 22 September 2026. Reproduce all of it with `scripts/verify-routing.sh`.

03 @Controller does not mean what it means in Spring

Spring has two annotations where NestJS has one. @RestController is @Controller with @ResponseBody already applied, and that second half is what makes your return value the response body.

Take it away and Spring does something else entirely: a plain @Controller returning the string "hello" hands that string to the view resolver as the NAME OF A TEMPLATE. The client gets an empty body.

NestJS has exactly one @Controller decorator and it behaves like Spring's @RestController. So if you are reading Spring examples while writing Nest, the one you copy decides whether your endpoint returns anything at all.

PlainController.java, which returns nothing to the client

```
@Controller // note: no @ResponseBody
class PlainController {
  @GetMapping("/x")
  String hello() {
    return "hello"; // a VIEW NAME, not a response
  }
}
```

Measured: the response body comes back empty.

payments.controller.ts, the only @Controller Nest has

```
@Controller('payments')
export class PaymentsController {
  @Get()
  hello() {
    return 'hello'; // the RESPONSE BODY
  }
}
```

@RestController is a convenience annotation combining @Controller and @ResponseBody.

Spring Framework documentation, Web MVC annotated controllers, checked 2026-09-22.

A NestJS controller method return value is serialised as the response body.

NestJS documentation, Controllers and HTTP adapters, checked 2026-09-22.

Plain @Controller: body empty. @RestController: body hello. NestJS @Controller: body hello.

Run on a real machine, 2026-09-22, by scripts/verify-routing.sh in the course repository.

04 The order you wrote the handlers in changes the answer

Two handlers. One maps to :id, one to the literal word recent, and the parameter one is written first, because that is the order things come to mind in.

In Spring that order is irrelevant: it picks the most specific pattern, so /payments/recent always reaches recent(). On a default NestJS setup the handlers are tried in declaration order, :id matches the word recent, and it wins.

Status 200 on both. /payments/123 still works on both. Nothing in your test output goes red.

payments.controller.ts, the bug

```
@Controller('payments')
export class PaymentsController {
  @Get(':id')
  byId(@Param('id') id: string) {
    return {handler: 'byId', id};
  }

  @Get('recent')
  recent() {
    return {handler: 'recent'};
  }
}
```

the fix, and it is one line of ordering

```
@Get('recent')    // literal routes first
recent() {}

@Get(':id')       // parameter routes second
byId() {}
```

This works whichever adapter you are on, which is why it is the rule worth keeping.

Spring selects the most specific matching pattern regardless of declaration order.

Spring Framework documentation, Web MVC annotated controllers, checked 2026-09-22.

Express matches in declaration order, so the first matching route wins.

Express routing documentation, checked 2026-09-22.



05 And that is not a fact about NestJS

"NestJS matches in declaration order" is a rule you will carry into the next project, and it is false.

NestJS does not have a router. It delegates to whatever HTTP library it sits on, and the default is Express. Express keeps an ordered stack, so a parameter route above a literal one swallows it.

Run the exact same controller, in the exact same order, on the Fastify adapter and you get the Spring answer, because Fastify's router prefers a static segment. Same file, same order, two different handlers, decided by an adapter choice somebody made in main.ts.

```
● ● ● GET /payments/recent, one controller, two adapters
```

```
Express  {"handler":"byId","id":"recent"}
Fastify  {"handler":"recent"}
```

Ordered stack against static segment first. Neither is wrong; they are different routers.

NestJS delegates HTTP handling to an underlying adapter, Express by default, with Fastify available.

NestJS documentation, Controllers and HTTP adapters, checked 2026-09-22.

Fastify uses a radix tree router that prefers static segments over parametric ones.

Fastify routing documentation, checked 2026-09-22.

The same controller answers with a different handler on each adapter.

Run on a real machine, 2026-09-22, by scripts/verify-routing.sh in the course repository.

06 The status code nobody typed

In Spring a handler returns 200 unless you say otherwise, whatever the verb. In NestJS the default is PER VERB. A GET returns 200. A POST returns 201, with nothing declared anywhere in your code.

So a faithful port silently changes the contract, and every client asserting on 200 breaks.

This is also where ResponseEntity went. You do not need it: return the object and the framework serialises it. When the envelope genuinely matters, put @HttpCode on the handler and say the number out loud.

nothing here declares a status code

```
@Post()
create(@Body() body: CreatePaymentDto) {
  return this.payments.create(body); // → 201
}
```

and the one line that puts the contract back

```
@Post()
@HttpCode(200)
create(@Body() body: CreatePaymentDto) {
  return this.payments.create(body); // → 200
}
```

No ResponseEntity, no wrapper object. Return the payload and declare the number.

NestJS responds 200 by default except for POST, which responds 201.

NestJS documentation, Controllers and HTTP adapters, checked 2026-09-22.

Measured: Spring POST 200, NestJS POST 201, NestJS POST with @HttpCode(200) 200.

Run on a real machine, 2026-09-22, by scripts/verify-routing.sh in the course repository.

Before you port a Spring controller

1. Declare literal routes above parameter routes, whichever adapter you are on
2. Check which adapter main.ts bootstraps before trusting any routing rule
3. Assume every POST now answers 201 until you have said otherwise
4. Replace ResponseEntity with a returned object, plus @HttpCode when the number matters
5. Remember NestJS @Controller behaves like Spring @RestController, not like @Controller
6. Write one request per handler that asks for the LITERAL route, not just the id route

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



X

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Node.js is a trademark of the OpenJS Foundation. Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. NestJS, Express and Fastify are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-22 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.